Università degli Studi di Messina
Dipartimento MIFT - Corso di Laurea Triennale in Informatica (L-31)

Laboratorio di Reti e Sistemi Distribuiti

HANDS-ON 5:
Analisi e Implementazione ICMP con Socket Raw

Massimo Mantineo
Matricola: 541924

Messina, A.A. 2024/2025

Indice

1 Problem Statement

L'obiettivo della quinta esercitazione (Hands-On 5) è lo studio e l'implementazione del protocollo **ICMP** (*Internet Control Message Protocol*) tramite l'impiego dei **Socket Raw**. L'attività consiste nel progettare un applicativo C capace di generare pacchetti *Echo Request* (Ping) personalizzati, bypassando gli stack di trasporto standard (TCP/UDP) del kernel. La sfida tecnica risiede nella gestione manuale dell'incapsulamento IP, nel calcolo del checksum secondo gli standard RFC e nel parsing accurato della risposta catturata dallo stream di bit grezzi proveniente dalla rete.

2 Stato dell'Arte

Il protocollo ICMP (RFC 792) è lo strumento fondamentale per la diagnostica e il controllo degli errori nelle reti IP.

- **Struttura del Messaggio**: Un pacchetto ICMP è composto da un header di 8 byte (Tipo, Codice, Checksum, Identificatore e Numero di Sequenza) seguito da un payload opzionale.
- **Socket Raw (`SOCK_RAW`)**: A differenza dei socket tradizionali, i socket raw permettono l'accesso diretto ai protocolli del livello di rete. L'uso di `IPPROTO_ICMP` istruisce il kernel a non aggiungere header di trasporto, delegando all'applicazione la costruzione del pacchetto. Per ragioni di sicurezza, queste operazioni richiedono privilegi di amministrazione (`root/sudo`).
- **Endianness**: Poiché i campi dell'header (come l'Identifier) sono multi-byte, è essenziale gestire la conversione tra l'ordine dei byte dell'host (Little-Endian su x86) e quello della rete (Big-Endian) tramite `htons()` e `ntohs()`.

3 Metodologia ed Implementazione

L'implementazione si articola nella costruzione del pacchetto e nell'analisi della risposta tramite aritmetica dei puntatori.

3.1 Costruzione del Pacchetto Echo Request

Viene allocato un buffer in cui si mappano le strutture `struct icmphdr`. Il checksum deve essere calcolato sull'intero header ICMP e sul payload prima dell'invio.

Snippet Calcolo Checksum ICMP

```
unsigned short compute\_checksum(void *addr, int len) {
    int sum = 0; unsigned short *ptr = addr;
    while (len > 1) { sum += *ptr++; len -= 2; }
    if (len == 1) sum += *(unsigned char *)ptr;
    sum = (sum >> 16) + (sum & 0xFFFF);
    sum += (sum >> 16);
    return (unsigned short)(~sum);
}
```

3.2 Ricezione e Parsing IP

Al momento della ricezione, il kernel restituisce anche l'header IP (20 byte). È necessario estrarre la lunghezza dell'header IP (`ihl`) per individuare l'inizio del payload ICMP:

Snippet Parsing Risposta

```
struct iphdr *ip\_hdr = (struct iphdr *)recv\_buffer;
// L'header ICMP inizia dopo (IHL * 4) byte
struct icmphdr *icmp\_reply = (struct icmphdr
    *)(recv\_buffer + (ip\_hdr->ihl * 4));
```

3.3 Istruzioni di Esecuzione

L'uso dei raw socket richiede privilegi elevati (sudo):

Build Run

```
$ gcc ICMP\_raw.c -o icmp\_ping
$ sudo ./icmp\_ping 8.8.8.8
```

4 Risultati

La validazione è stata condotta interrogando il server DNS di Google (`8.8.8.8`) con privilegi di `root`.

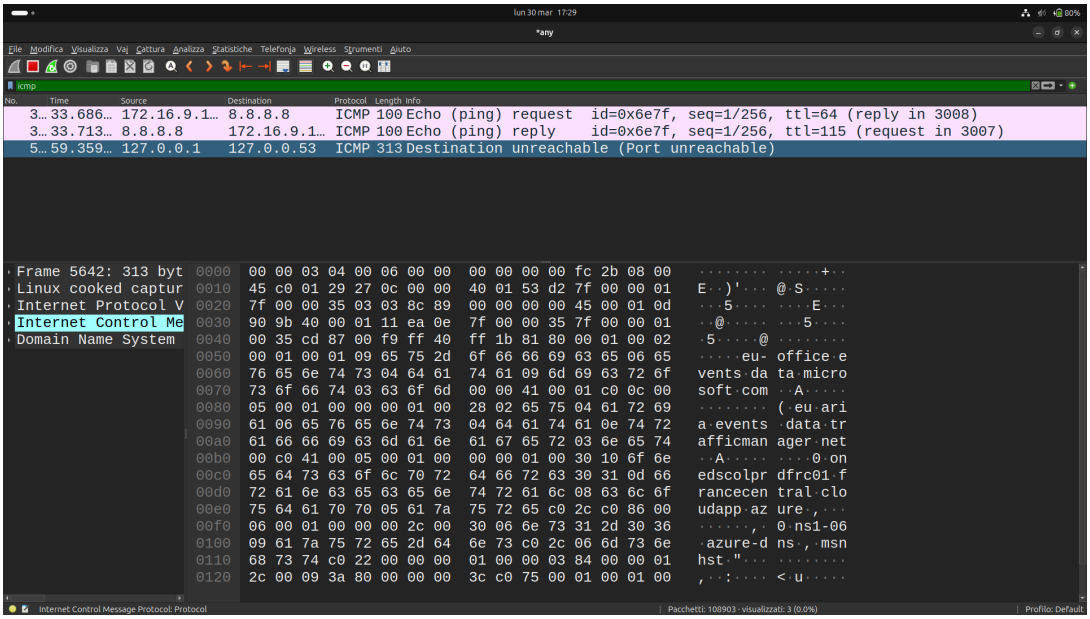

Figure 1: Cattura pacchetti ICMP su Wireshark.

4.1 Test di Raggiungibilità

Il programma ha riportato con successo la ricezione dell'Echo Reply (Tipo 0), confermando il corretto matching tra l'ID del processo e l'ID contenuto nel pacchetto di ritorno.

Output Ping Custom

```
$ sudo ./icmp\_ping 8.8.8.8
Invio ICMP Echo Request (id=1234, seq=1)...
Ricevuta risposta: seq=1, TTL=117, bytes=64
```

4.2 Analisi Wireshark

L'ispezione della cattura (Figura 5) conferma che il payload personalizzato 'AAAA' (`0x41`) è stato correttamente incapsulato e restituito dal server remoto, validando la funzione di checksum.

5 Conclusioni e Sviluppi Futuri

L'Hands-On 5 ha permesso di comprendere l'anatomia dei protocolli di controllo e il potere derivante dall'accesso diretto al network stack tramite socket raw. Questa competenza è la base per lo sviluppo di tool di scansione e monitoraggio professionale.
Sviluppi futuri:

- **Traceroute**: Implementazione di un tool di tracciamento rotta manipolando il campo `TTL` (Time-To-Live) dell'header IP per indurre errori di *Time Exceeded*.

- **ICMPv6**: Adattamento del codice per lo stack IPv6, analizzando le differenze nella gestione del Checksum (che in IPv6 include uno pseudo-header IP).